

# FPGA Design Flow

From RTL Code to a Running LED Blinker on Hardware (Xilinx Vivado)

## 1. Write / Add Your RTL Source

Create your Verilog or VHDL file (e.g. top.v) containing your logic — such as a clock divider counter and LED output for a blinker circuit. Add it to your project via Add Sources in the Flow Navigator.

Tip: Keep your top-level module's port names clear (clk, led, rst) — you'll reference them directly in your constraints file.

## 2. Add Constraints (XDC File)

Map your design's ports (clk, led, etc.) to actual physical pins on the board, and specify the input clock frequency. Without this step, Vivado has no way of knowing which physical pin the LED or clock signal connects to.

Tip: Most dev boards (Basys 3, Nexys, etc.) publish a master XDC file online with all pins pre-mapped and commented out — just uncomment the ones you use.

## 3. Run Synthesis

Vivado translates your RTL description into a generic netlist of logic gates and flip-flops that could, in principle, be implemented on any FPGA. This checks your code is synthesizable and reports resource usage.

Look for: “synth\_design completed successfully” with 0 critical warnings and 0 errors.

## 4. Run Implementation

This stage places the synthesized logic onto specific physical locations within the target FPGA's fabric (place) and wires up the connections between them using the chip's routing resources (route). Timing analysis is run to confirm your design can meet its clock speed requirements.

Look for: “All user specified timing constraints are met” in the Timing report, with 0 failing endpoints on Setup, Hold, and Pulse Width.

## 5. Generate Bitstream

Vivado converts the fully placed-and-routed design into a .bit file — a binary configuration stream that tells the FPGA's physical hardware exactly how to configure every logic block, flip-flop, and routing switch to become your circuit.

This is a single click in the Flow Navigator under “Program and Debug.” It typically takes under a minute for a small design like an LED blinker.

## 6. Open Hardware Manager & Connect

Open the Hardware Manager and use Open Target → Auto Connect to detect your FPGA board over its USB/JTAG programming cable.

If the board isn't detected, check the USB cable, confirm board drivers are installed, and that no other program (like a serial terminal) is holding the port.

## 7. Program the Device

Click Program Device, select your generated .bit file, and Vivado loads the configuration onto the FPGA. Your logic begins running immediately — you should see the LED blinking at the rate your counter design produces.

Note: On most boards this configuration is stored in volatile memory, so it resets on power loss and must be reprogrammed (unless you flash it to onboard non-volatile memory).

## 8. Debug If Needed

If the LED behaves unexpectedly (e.g. blinking too fast, staying dark, or staying lit), revisit your RTL. Common culprits: a counter that's too narrow for the desired blink rate, selecting the wrong bit of the counter to drive the LED, or an incorrect assumed clock frequency. Fix the code, then re-run synthesis through programming.

Tip: To blink once per second from a 100 MHz clock, you need a counter of about 27 bits ( $2^{27} \approx 134\text{M}$ ) and should drive the LED from the counter's most significant bit.

Summary: Code → Constraints → Synthesis → Implementation → Bitstream → Program → Debug → Repeat